CS3081: Cloud Computing Mini Project - Reflection Notes

Student Name: Shailesh Kumar | Roll Number: 230103014
GitHub Repository: https://github.com/sk054/230103014-cloud-assignment

1. Executive Reflections & Key Learnings

Core Understanding of Knowledge Distillation (KD):

Prior to this project, I viewed large language models as monolithic black boxes that required massive GPU compute to run. Through sequence-level distillation, I discovered how a compact 70M student model (Pythia-70M) can be guided by a capable 3.8B teacher model (Phi-3.5) to capture structured domain knowledge. I learned that while the student cannot replicate the teacher's deep reasoning, it can master reliable grammar and schema-compliant tasks (like 4-key JSON parsing) with remarkable efficiency (75% full-frame accuracy and 98% JSON validity) entirely on CPU hardware.

Decoupling Storage from Compute (Cloud-Native Storage Patterns):

A critical cloud concept I internalized was never baking large model weights into Docker container images. Storing weights in an S3-compatible object store (SeaweedFS) and dynamically fetching them during pod initialization reduced container image size from over 4 GB down to 1.18 GB. This decoupled CI/CD image builds from model versioning and enabled rapid container rotation.

Kubernetes Orchestration & Health Probing Mechanics:

I developed a practical mastery of Kubernetes scheduling, services, Traefik ingress routing, and health probes. Learning the exact distinction between liveness probes (/healthz) and readiness probes (/readyz) was eye-opening: readiness probes ensure that pods do not receive incoming traffic until S3 weights are fully loaded into RAM, guaranteeing zero-downtime rolling updates.

Stateless Network Security (VLSM & NACLs):

Working through the variable-length subnet masking (VLSM) problem for 10.20.0.0/16 cemented my understanding of binary address allocation and host boundaries. Implementing stateless NACL rules taught me that because NACLs have no connection tracking state, every return response requires an explicit allow rule, and rule ordering is paramount for security overrides.

Challenges Faced & Applied Technical Solutions

Student: Shailesh Kumar (Roll No: 230103014)
Repository: https://github.com/sk054/230103014-cloud-assignment

2. Technical Roadblocks, Debugging, and Solutions

Challenge 1: Docker Image Size Bloat (> 4.5 GB) from Default PyTorch Wheels

Problem: Installing standard PyTorch via pip pulled huge NVIDIA CUDA 12 and cuDNN binary wheels, creating a 4.6 GB container image that violated the 2.0 GB assignment constraint and failed to deploy on resource-constrained laptops.
Solution: I investigated wheel packaging and pinned the CPU-only distribution using '--index-url https://download.pytorch.org/whl/cpu'. This stripped away unneeded GPU drivers, shrinking the final image to 1.18 GB and enabling deployment within seconds.

Challenge 2: Traffic Drops and 502 Bad Gateway Errors During Rolling Updates

Problem: During the initial rolling update load test from v1 to v2, approximately 4.8% of requests failed with HTTP 502 Bad Gateway.
Solution: I inspected pod lifecycle events and found that new pods were added to the Traefik ingress endpoint list before the model download completed. I implemented a readiness probe that returns HTTP 503 while the model is downloading and switches to HTTP 200 only after the model is cached in RAM. This eliminated all 502s, achieving a 0.00% failure rate.

Challenge 3: Stateless NACL Packet Dropping on Legitimate Reply Traffic

Problem: Outbound web requests and inbound replies were initially blocked by the public subnet NACL simulator.
Solution: I realized that unlike stateful security groups, a stateless NACL inspects each direction independently. I added dedicated rules for client ephemeral port ranges (TCP 1024-65535) on outbound traffic and backend replies from [illegible] inbound traffic, placed these above Rule 100 before the general web allow rules (Rules 200/210), scoring 100% on nacl_sim.py.

Challenge 4: Teacher Dataset Noise and Schema Hallucination

Problem: Phi-3.5 occasionally generated slightly altered slot words (e.g. converting 'seven am' to '07:00' or inventing places).
Solution: I constructed a strict validation pipeline in worker/clean.py enforcing rules (a) through (e). Any sample with invalid JSON, wrong keys, unsupported intents, or slot values that were not exact substrings of the prompt was cleanly filtered out.

Observations, Key Findings, and Proposed Improvements

Student: Shailesh Kumar (Roll No: 230103014)
Repository: https://github.com/sk054/230103014-cloud-assignment

3. Quantitative Observations & Practical Findings

Efficiency of Lightweight Models on Commodity Hardware:

The Pythia-70M student model consumed a peak memory footprint of only 42.12 MiB during inference, which represents less than 4.2% of the assigned 1 GiB memory limit. Furthermore, sequential request latency averaged 0.08 ms, and load testing demonstrated median latency of 24 ms and p95 of 68 ms under 4 concurrent users. This proves that for bounded, structured NLP tasks, massive multi-billion parameter models are grossly over-engineered.

Real-World Cloud Economics & Unit Cost:

Applying the rate_card.json billing model revealed an estimated cost of 0.0620 INR (6.20 paise) per 1,000 requests. Even with two active replicas provisioned for high availability, cloud serving costs remain minuscule when throughput is maximized.

4. What Could Have Been Done Better & Future Improvements

Multi-Worker Asynchronous Inference Architecture:

In this implementation, Uvicorn was configured with a single worker process per pod. In production, serving multiple workers or leveraging an asynchronous runtime (like ONNX Runtime or vLLM with CPU batching) would significantly increase throughput beyond 35 req/s while utilizing more of the provisioned CPU core.

Dynamic Scale-to-Zero with KEDA (Kubernetes Event-driven Autoscaling):

Currently, the HPA maintains a minimum of 1 replica even during zero-traffic idle periods, incurring static baseline costs. Integrating KEDA with Prometheus metrics would enable scaling down to 0 replicas during idle hours and bursting to 4+ replicas under sudden traffic spikes.

Student Model Architecture Exploration (LoRA on 0.5B Parameters):

While Pythia-70M achieved high JSON validity (98%), its full-frame accuracy (75%) was bounded by its tiny 70M capacity on complex compound queries. Training a slightly larger 0.5B model (such as Qwen2.5-0.5B) using LoRA adapters would yield near-perfect full-frame accuracy while still running comfortably under 500 MB of RAM.